

Onderzoeksrapport Plattegrondherkenning



Inhoudsopgave

Inhoudsopgave	2
Inleiding	3
Bestanden	3
DXF / DWG	3
PDF.....	3
PNG / JPG	3
Conclusie bestanden	4
Het Project	4
React Native	4
PDF Handling.....	4
Parsing.....	5
Data berekening.....	5
Resultaten opslaan	5
Pijplijn	5
Alternatieve bestandformaten.....	6
Richting.....	6
Conclusie	6



Inleiding

Maeconomy wilt nauwkeurige data meten vanuit gebouwen en deze in een gestructureerde vorm opslaan. Zo kan Maeconomy deze data gebruiken om municipaliteiten en bedrijven te informeren over potentiële recycling van grondstoffen/materialen van gebouwen.

Maeconomy wilt deze data uit al-bestaande plattegronden kunnen halen en kunnen verwerken. Dit document zal de opties verkennen waarin Maeconomy dit idee verder kan opbouwen, de voor- en nadelen en de knopen die kunnen verwacht worden met dit project.

Bestanden

Het begint allemaal bij de bestanden. Als de bestanden die wij willen gebruiken niet van pas zijn, kunnen wij niet verder met het opnemen van de data in het bestand. Hiervoor moeten wij plannen en voorbereid zijn. Er zijn meerdere opties, maar als start-up is het belangrijk voor Maeconomy om een zo breed mogelijk omvang te hebben.

DXF / DWG

Het format voor CAD Vector bestanden. Dit format heeft elk exact coördinaat van elk lijntje dat in het bestand zit. Dit houdt in dat er geen gok-werk zal zitten en dat alle beschikbare data zich gelijk laat zien. Het probleem met dit format is dat het ongebruikelijk is om toegang te krijgen tot deze bestanden. Het is direct, maar zal niet je eerste keuze zijn.

PDF

PDFs zijn in principe de “golden standards” voor schematica exports. Hier is de data ook geëxporteerd in een digitale manier, dus wederom geen gok-werk met het afmeten van het document/schematica. Dit format zal prioriteit nummer een moeten zijn voor het project en de basis voor het parsen van bestanden.

PNG / JPG

Niet elk gebouw zal toegankbare bestanden hebben of uitgebreide documenten. Hiervoor zullen we een OCR fallback kunnen maken. Dit is een intens “last resort” en zal tijd nodig hebben voor ontwikkeling, maar zal een hele sterke fallback kunnen zijn mits de Andere twee bestandtypes geen optie zijn.



Conclusie bestanden

Nu dat we weten wat voor bestandtypes kunnen verwachten en waarmee we waarschijnlijk zullen zitten, kunnen wij hierop plannen en ingaan. Het is duidelijk dat PDF bestanden onze ‘hoofdbron’ zal zijn voor data parsing, dus, hier zullen we ons op ook richten. We zullen ook een klein kijkje nemen naar oplossingen voor de andere bestand types.

Het Project

We weten nu wat voor bestanden ons te wachten staat. Laten we ons eerst focus leggen op PDF bestanden zoals eerder besproken. Hier zullen we uitbreiden over wat Maeconomy nodig zal hebben om dit project in productie te zetten en waar we vast kunnen lopen tijdens ontwikkeling. Om het project zo simpel mogelijk te houden, zullen we een webapp maken dat zich prioritiseert op desktop gebruik en verder uitbreiden naar mobiele toepassing.

React Native

Maeconomy’s hoofdtal is React Native. Deze framework is gericht op simple UI design en een “HTML” stijl van bouwen. Het gebruikt Javascript, dus het is makkelijk om in te komen en aan te werken als een buitenstaander of junior dev. Verder maakt React Native cross-platform development veel soepeler, dus kan de app ook gebouwd worden voor mobiele toepassingen. Met React zal je wel veel werken met 3rd party libraries, dus dit kan wel veel meer problemen veroorzaken met versiebeheer of compatibility.

PDF Handling

Onze eerste stap zal zijn om PDF bestanden te kunnen weergeven binnen onze app. Gelukkig heeft React al een library hiervoor genaamd: ‘**react-native-pdf**’. Verder moeten we het “rauwe bestand” houden om informatie uit het bestand te kunnen halen. ‘**react-native-blob-util**’ kan hiermee helpen.



Parsing

Parsing houdt in dat je data opneemt en dit meer leesbaar maakt voor algemeen man. React Native kan dit niet uit zichzelf, dus zullen we een backend hiervoor moeten maken. ‘pdf-parse’ kan onze oplossing hiervoor zijn. Met pdf-parse kunnen we text, data en afbeeldingen uit bestanden halen. Python met ‘pdfplumber’ kan hier ook mee helpen mits pdf-parse niet de benodigde functionaliteiten geeft.

Data berekening

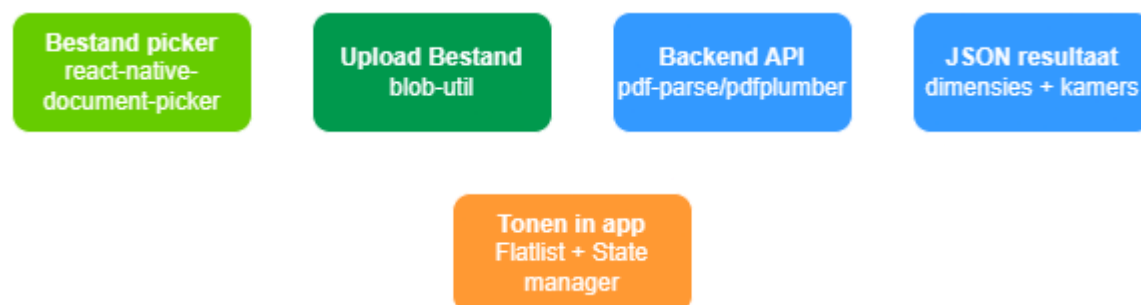
Eenmaal de data is opgenomen uit een document, moeten we de correcte berekeningen toepassen om ervoor te zorgen dat de opmetingen kloppen. Dit kunnen we doen door simpele formules in de app zelf bij te houden, zoals; ‘L x B x H’. Voor hele aparte gevallen kunnen we altijd een AI call hebben als fallback. Verder moeten we ook de schaal kunnen opnemen van een plattegrond, zoals 100:1, anders klopt de data van het gebouw niet en slaan we verkeerde data op. Tijdens het berekenen van maten, zullen we automatisch scannen voor een schaal binnen het PDF bestand, mits deze niet aanwezig is, kunnen we de user dit zelf laten toevoegen als fallback.

Resultaten opslaan

Maeconomy heeft al een structuur om data op te slaan die we opmeten/opnemen van schematica. Deze heet de “Internet of Materials”. Dus hier hebben we al een goed begin in. We zullen eers focusen op local-only ontwikkeling totdat we een folderstructuur hebben waarmee we blij zijn.

Pijplijn

Nu dat we een simpele structuur hebben voor het handelen van bestanden, kunnen we dit weergeven met een diagram om het idee thuis te brengen.



Alternatieve bestandformaten

Voordat we verder gaan, moeten we natuurlijk kijken naar de andere twee bestandtypes die we vast zullen tegenmoeten komen. Met het parsen van DXF / DWG kun je ‘**ezdxf**’ met Python gebruiken en de directe data uit het bestand halen. Voor puur foto bestanden zal het veel complexer moeten worden helaas. Eerst zouden we de foto moeten “deskewen” oftewel; rechtzetten. Daarna door een OCR (Optical Character Recognition) engine zoals Tesseract, EasyOCR of Doctr. Inhoudelijk vanuit beide punten zou het berekenen van het structuur hetzelfde blijven met formules. Het enige wat zou veranderen is de manier waarop wij de data uit verschillende bestanden halen.

Richting

Het belangrijkste voor dit project is benaderbaarheid. Dit houdt in dat we ons moeten richten om het proces van bestand uploaden, tot het uithalen van de data zo soepel mogelijk blijft zodat er zo min mogelijk ‘human error’ in komt te zitten. Stappen uit de handen halen van de user dus. Dit kunnen we bereiken door zo veel mogelijk feedback te geven aan de user tijdens de berekeningen en een visueel te geven na berekeningen. Hoewel dit een prioriteit zal zijn, is het grootste prioriteit natuurlijk de data uit bestanden halen. Als dit eenmaal concreet is, kunnen we het project richting UI/UX richten.

Conclusie

De bouwstenen (woordspeling inbegrepen) en libraries liggen voor Maeconomy om dit project voort te zetten naar een gereedschap dat het bedrijf kan laten opvallen. Als wij kunnen laten zien dat informatie uit bestanden halen ons expertise is, zullen we veel meer opvallen op de radar van bedrijven. React en mits nodig Python zijn perfect voor onze use case en zijn met voordeel makkelijk te benaderen.

